TP 10 : Application Android avec Firebase Realtime Database

Développement Mobile - BTS

Objectif du TP

Réaliser une application Android permettant :
— D'ajouter un message avec le nom de son auteur
— D'enregistrer les messages dans Firebase
— D'afficher les messages dans un RecyclerView
— De comprendre le rôle des règles de sécurité Firebase

1 Partie 1 : Création du projet Android

— Créer un nouveau projet Android (Empty Views Activity)
— Nom du projet : **TP10_Firebase**
— Langage : Java
— Minimum SDK : API 24

2 Partie 2 : Connexion à Firebase

Il existe **deux méthodes** pour connecter une application Android à Firebase.

Méthode 1 : Connexion automatique

— Ouvrir Android Studio
— Menu : Tools → Firebase
— Sélectionner Realtime Database
— Cliquer sur Connect to Firebase
— Suivre l'assistant
Android Studio :
— Ajoute le fichier google-services.json
— Ajoute les dépendances automatiquement

Méthode 2 : Connexion manuelle

— Aller sur `https://console.firebase.google.com`
— Créer un nouveau projet Firebase
— Ajouter une application Android
— Télécharger le fichier google-services.json

— Le placer dans le dossier app/
Puis ajouter dans build.gradle (Module) :

```
implementation 'com.google.firebase:firebase-database'
```

Et dans build.gradle (Project) :

```
classpath 'com.google.gms:google-services:4.3.15'
```

3 Partie 3 : Configuration des règles Firebase

IMPORTANT : Sans règles correctes, l'écriture ne fonctionnera pas.
Dans Firebase Console → Realtime Database → Rules :

```
{
  "rules": {
    ".read": "now < 1773705600000",
    ".write": "now < 1773705600000"
  }
}
```

Cela signifie :
— Autoriser lecture et écriture jusqu'au 17 mars 2026
— Après cette date, l'accès sera refusé
Attention : Ces règles sont uniquement pour le développement (mode test).

Test de la connectivité

```
    FirebaseDatabase database = FirebaseDatabase.getInstance();
        DatabaseReference myRef = database.getReference("message");

        myRef.setValue("Hello, World!");
```

Vérifiez sur Firebase si votre message a été bien envoyé !

4 Partie 4 : Structure du modèle

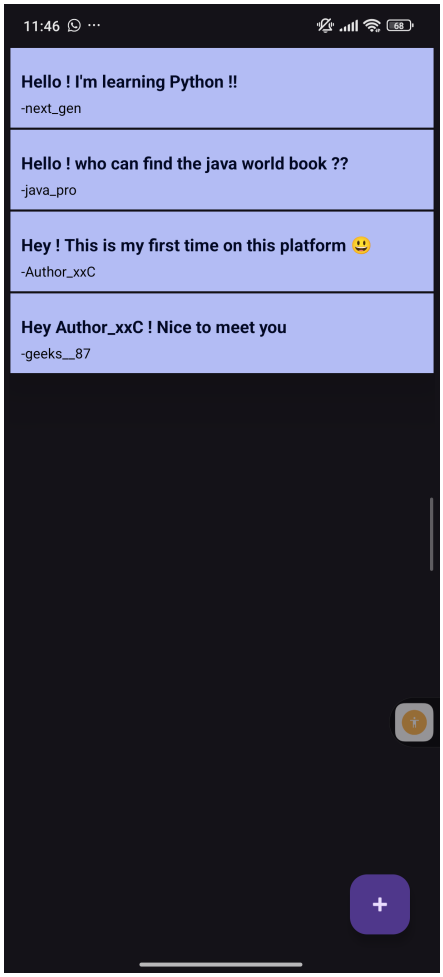
4.1 Structure générale du projet

Après la création du projet Android et la connexion à Firebase, l'arborescence principale du projet est la suivante :

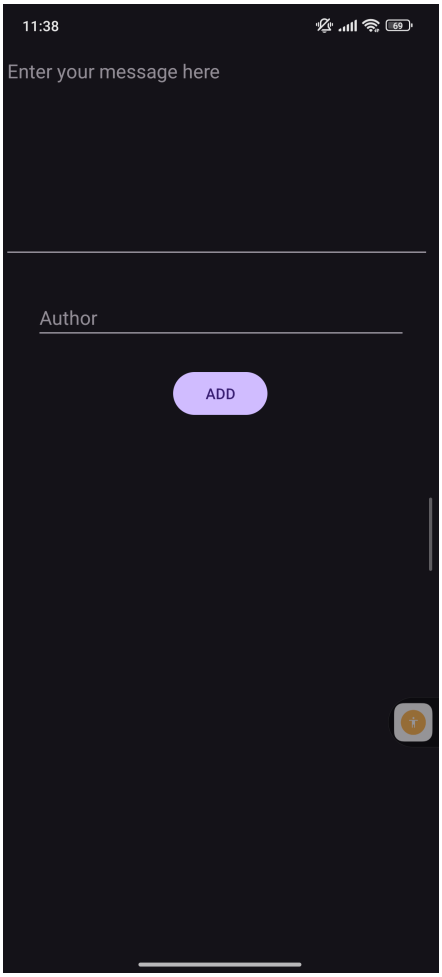
— **java/com.example.tp9_firebase**
  — MainActivity.java
  — AddMessage.java
  — Message.java
  — MsgAdapter.java
  — MsgViewHolder.java
— **res/layout**
  — activity_main.xml
  — activity_add_message.xml
  — msgitem.xml

Rôle de chaque fichier

- **MainActivity** : Affiche la liste des messages avec un RecyclerView et récupère les données depuis Firebase.
- **AddMessage** : Permet à l'utilisateur de saisir un message et son nom, puis de l'enregistrer dans Firebase.
- **Message** : Classe modèle représentant un message (clé, contenu, auteur).
- **MsgAdapter** : Adaptateur du RecyclerView permettant d'afficher les messages.
- **MsgViewHolder** : Classe responsable de la liaison entre les composants graphiques (TextView) et les données.
- **activity_main.xml** : Interface principale contenant le RecyclerView et le bouton d'ajout.
- **activity_add_message.xml** : Interface permettant d'ajouter un message.
- **msgitem.xml** : Modèle graphique d'un élément affiché dans le RecyclerView.



(a) Liste RecycleView contenant les données



(b) Ajout d'un nouveau message

FIGURE 1 – Interfaces de l'application

4.2 Structure du modèle (Classe Message)

La classe **Message** représente la structure des données enregistrées dans Firebase Realtime Database.

```java
public class Message {
    private String msg;
    private String key;
    private String author;

    public Message() {}

    public Message(String key, String msg, String author) {
        this.key = key;
        this.msg = msg;
        this.author = author;
    }

    public String getMsg() { return msg; }
    public String getAuthor() { return author; }
    public String getKey() { return key; }

    public void setMsg(String msg) { this.msg = msg; }
    public void setAuthor(String author) { this.author = author; }
    public void setKey(String key) { this.key = key; }
}
```

Pourquoi un constructeur vide est obligatoire ?

Firebase utilise la désérialisation automatique. Lors de la récupération des données :

```java
Message message = data.getValue(Message.class);
```

Firebase crée l'objet en utilisant le constructeur vide, puis remplit les attributs via les setters.

Sans constructeur vide, la lecture des données échoue.

Structure des données dans Firebase

Dans Realtime Database, les messages sont stockés ainsi :

```
msg
              -Nkfj39dkd93
                      msg : "Bonjour"
                      author : "Amina"
                      key : "-Nkfj39dkd93"
              -Nkfj40abc22
                    msg : "Hello"
                    author : "Ali"
                    key : "-Nkfj40abc22"
```

Chaque message possède :

- Une clé unique générée avec push()
- Le contenu du message
- Le nom de l'auteur

5 Partie 5 : Logique d'ajout d'un message

5.1 Objectif

Permettre à l'utilisateur :
- De saisir un message
- De saisir son nom
- D'enregistrer les données dans Firebase

5.2 Récupération des données saisies

Dans AddMessage.java :

```
text = findViewById(R.id.editTextText);
text2 = findViewById(R.id.editTextText2);
btn = findViewById(R.id.button);
btn.setOnClickListener(v->{
    String msg = text.getText().toString().trim();
    String author = text2.getText().toString().trim();
    if (msg.isEmpty()) {
        text.setError("Champ␣vide␣!");
        return;
    }
    if (author.isEmpty()) {
        text2.setError("Champ␣vide␣!");
        return;
    }
    addMsgDB(msg, author);
});
```

Important :
- trim() supprime les espaces inutiles
- Toujours vérifier si les champs sont vides

5.3 Génération d'une clé unique

```
DatabaseReference ref = database.getReference("msg");
String key = ref.push().getKey();
```

Pourquoi push() est important ?
- Génère une clé unique
- Évite l'écrasement des données
- Permet l'ajout de plusieurs messages

Sans push(), chaque nouveau message remplacerait le précédent.

5.4 Enregistrement dans la base

```
ref.child(key).setValue(message);
```

Explication détaillée de child()

La méthode **child()** permet de naviguer dans l'arborescence de Firebase Realtime Database.

Si l'on écrit simplement :

```
ref.setValue(message);
```

Alors le contenu de **msg** serait entièrement remplacé par un seul message.

En revanche, avec :

```
ref.child(key).setValue(message);
```

nous disons à Firebase :
— Aller dans le noeud **msg**
— Créer un sous-noeud portant le nom de la clé générée
— Enregistrer le message à cet emplacement précis
Chaque appel à **push()** génère une clé unique, et **child(key)** permet d'écrire dans ce sous-noeud.

La combinaison :

```
push() + child(key)
```

garantit :
— Un identifiant unique
— Aucun écrasement de données
— Une structure organisée
Flux réel :
Application → Firebase SDK → Serveur Firebase → Rules → Écriture
Si les règles sont incorrectes, l'écriture sera bloquée.

5.5 Importance des règles

Les règles utilisées dans ce TP :

```
{
  "rules": {
    ".read": "now␣<␣1773705600000",
    ".write": "now␣<␣1773705600000"
  }
}
```

Cela autorise l'écriture jusqu'au 17 mars 2026.
Sans ces règles, le code fonctionnerait mais Firebase refuserait l'accès.

5.6 Fonction addMsgDB

Dans AddMessage.java :

```
private void addMsgDB(String msg, String author) {
HashMap<String, Object> msgHashMap  =new HashMap<>();
msgHashMap.put("msg", msg);
msgHashMap.put("author", author);
FirebaseDatabase database = FirebaseDatabase.getInstance();
DatabaseReference msgRef = database.getReference("msg");
String key =  msgRef.push().getKey();
msgHashMap.put("key", key);
msgRef.child(key).setValue(msgHashMap).addOnCompleteListener(new
   OnCompleteListener<Void>() {
    @Override
    public void onComplete(@NonNull Task<Void> task) {
        Toast.makeText(AddMessage.this, "Enregistré", Toast.
           LENGTH_LONG).show();
    }
});
text.getText().clear();
text2.getText().clear();


}
```

6 Partie 6 : Lecture des messages en temps réel

6.1 Objectif

Afficher automatiquement tous les messages enregistrés.
Dans MainActivity.java

```
    private FloatingActionButton floatingActionButton;
private RecyclerView recyclerView;
private MsgAdapter msgAdapter;
private ArrayList<Message> msgList;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    // Permet  d utiliser  toute la surface de  l  écran (gestion
       moderne des bordures)
    EdgeToEdge.enable(this);

    // Association de l'activité avec son layout XML
    setContentView(R.layout.activity_main);

    // Récupération des composants graphiques depuis le layout
    floatingActionButton = findViewById(R.id.floatingActionButton);
    recyclerView = findViewById(R.id.recyclerView);
```

```
    // Initialisation de la liste locale qui va contenir les
       messages
    // Cette liste est temporaire et vit uniquement dans
        l application
    msgList = new ArrayList<>();

    // Création de  l adapter  en lui passant la liste
    //  L adapter  fera le lien entre les données (msgList) et le
       RecyclerView
    msgAdapter = new MsgAdapter(msgList);

    // Définition du type  d affichage  du RecyclerView
    // LinearLayoutManager     affichage vertical en liste
    recyclerView.setLayoutManager(new LinearLayoutManager(this));

    // Association de  l adapter  au RecyclerView
    recyclerView.setAdapter(msgAdapter);

    // Gestion du clic sur le bouton flottant
    // Ouvre l'activité d'ajout d'un message
    floatingActionButton.setOnClickListener(v -> {
        Intent i = new Intent(MainActivity.this, AddMessage.class);
        startActivity(i);
    });

    // Chargement des messages depuis Firebase
    // Cette méthode écoute la base de données en temps réel
    loadMessages();
}
```

Analyse pédagogique du code MainActivity

1) Pourquoi initialise-t-on la liste AVANT l'adapter ?

L'adapter reçoit une référence vers la liste de données :

```
msgAdapter = new MsgAdapter(msgList);
```

Si la liste **msgList** n'est pas initialisée avant cette ligne, l'application provoquera une erreur de type :

NullPointerException

La structure des données doit toujours être créée avant d'être utilisée par un composant graphique.

2) Pourquoi séparer msgList et msgAdapter ?

- **msgList** représente les données (logique métier)
- **msgAdapter** représente l'affichage (logique graphique)

Cette séparation respecte le principe fondamental :

Séparer les données de leur représentation

Cela permet :
— Une meilleure organisation du code
— Une maintenance plus simple
— Une architecture plus claire

3) Pourquoi loadMessages() est-il appelé à la fin ?

La méthode **loadMessages()** est appelée après :
— L'initialisation du RecyclerView
— L'attachement de l'adapter
Si l'on appelait loadMessages() avant l'attachement de l'adapter, la méthode :

```
adapter.notifyDataSetChanged();
```

ne fonctionnerait pas correctement.
Conclusion : L'interface doit être prête avant de charger les données.

4) Comprendre le flux réel d'exécution

Le fonctionnement interne suit le schéma suivant :

onCreate()
↓
Interface prête
↓
loadMessages()
↓
Firebase renvoie les données
↓
msgList est remplie
↓
notifyDataSetChanged()
↓
RecyclerView s'actualise

Firebase agit comme un serveur distant. Le RecyclerView ne se met à jour que lorsque l'adapter est explicitement notifié d'un changement.

6.2 Écoute des modifications

```
    private void loadMessages() {
        DatabaseReference myRef = FirebaseDatabase
                .getInstance()
                .getReference("msg");
```

```
        myRef.addValueEventListener(new ValueEventListener() {
            @Override
            public void onDataChange(@NonNull DataSnapshot
               dataSnapshot) {
                // This method is called once with the initial
                   value and again
                // whenever data at this location is updated.
                msgList.clear();
                for (DataSnapshot data : dataSnapshot.getChildren()
                   ) {
                    Message value = data.getValue(Message.class);
                    msgList.add(value);
                }
                msgAdapter.notifyDataSetChanged();

            }

            @Override
            public void onCancelled(DatabaseError error) {
                // Failed to read value
                String TAG = "";
                Log.w(TAG, "Failed␣to␣read␣value.", error.
                   toException());
            }
        });

    }
```

6.3 Analyse importante

addValueEventListener() :
— Écoute les changements en temps réel
— Se déclenche à chaque modification
— Recharge la liste automatiquement
messageList.clear() est obligatoire, sinon les données seraient dupliquées.
notifyDataSetChanged() informe le RecyclerView qu'il doit se redessiner.

7 Partie 7 : RecyclerView et affichage des messages

7.1 Rôle du RecyclerView

Le RecyclerView est un composant moderne permettant d'afficher une liste dynamique de données.

Contrairement au ListView :
— Il est plus performant
— Il recycle les vues automatiquement
— Il sépare clairement les données de l'affichage

Dans notre application, il permet d'afficher tous les messages récupérés depuis Firebase.

7.2 Architecture du RecyclerView

Le fonctionnement repose sur trois éléments fondamentaux :

— Une liste de données : **ArrayList<Message>**
— Un Adapter : **MsgAdapter**
— Un ViewHolder : **MsgViewHolder**

Cette architecture respecte le principe de séparation entre la logique métier et l'interface graphique.

7.3 Le ViewHolder

Le ViewHolder représente un seul élément graphique de la liste.

```
public class MsgViewHolder extends RecyclerView.ViewHolder {
    public TextView msgTv, authorTv;

    public MsgViewHolder(View itemView) {
        super(itemView);

        msgTv = itemView.findViewById(R.id.textView);
        authorTv = itemView.findViewById(R.id.textView2);
    }
}
```

Rôle :

— Associer les composants XML aux variables Java
— Éviter les appels répétés à findViewById()
— Optimiser les performances

7.4 L'Adapter

L'Adapter établit le lien entre les données et leur représentation graphique. Méthodes obligatoires dans l'Adapter :

— getItemCount()
— onCreateViewHolder()
— onBindViewHolder()

```
public class MsgAdapter
    extends RecyclerView.Adapter<MsgViewHolder> {

    private List<Message> messageList;

    public MsgAdapter(List<Message> messageList) {
        this.messageList = messageList;
    }
```

Méthode 1 : onCreateViewHolder()

```
@Override
public MsgViewHolder onCreateViewHolder(
    ViewGroup parent, int viewType) {

    View view = LayoutInflater
        .from(parent.getContext())
        .inflate(R.layout.msgitem, parent, false);

    return new MsgViewHolder(view);
}
```

Rôle :
— Créer la vue d'un élément
— Associer le layout msgitem.xml
— Retourner un ViewHolder
Cette méthode est appelée uniquement lorsqu'une nouvelle vue doit être créée.

Méthode 2 : onBindViewHolder()

```
@Override
public void onBindViewHolder(
    MsgViewHolder holder, int position) {

    Message msg = messageList.get(position);

    holder.msgTv.setText(msg.getMsg());
    holder.authorTv.setText("- " + msg.getAuthor());
}
```

Rôle :
— Récupérer le message correspondant à la position
— Remplir les TextView avec les données
Cette méthode est appelée à chaque fois qu'un élément doit être affiché ou réaffiché.

Méthode 3 : getItemCount()

```
@Override
public int getItemCount() {
    return messageList.size();
}
```

Rôle :
Indiquer au RecyclerView combien d'éléments doivent être affichés.

7.5 Flux réel d'affichage

Firebase renvoie données
↓
msgList est remplie
↓
notifyDataSetChanged()
↓
RecyclerView demande le nombre d'éléments
↓
getItemCount()
↓
Création ou recyclage des vues
↓
onBindViewHolder()
↓
Affichage à l'écran

Le RecyclerView ne stocke pas les données.
Il affiche uniquement ce que l'Adapter lui fournit.
La séparation est donc :
— Données : ArrayList<Message>
— Logique d'affichage : MsgAdapter
— Composant graphique : RecyclerView

8 Travail demandé

1. Implémenter l'ajout d'un message
2. Afficher les messages dans un RecyclerView
3. Tester avec plusieurs messages
4. Modifier les règles et observer le comportement

Questions de réflexion

1. Que se passe-t-il si les règles sont :

```
".read": false,
".write": false
```

2. Pourquoi Firebase bloque-t-il l'écriture même si le code est correct ?
3. Quelle différence entre connexion automatique et manuelle ?

Conclusion

Dans ce TP, nous avons appris :
— À connecter une application Android à Firebase

— À comprendre le rôle des règles de sécurité
— À stocker et afficher des données en temps réel
— À utiliser RecyclerView avec Firebase